Detección de ruido mediante DBSCAN: evaluación experimental de desempeño de una versión serial y dos paralelas con OpenMP

Proyecto Apertura · Cómputo Paralelo y en la Nube · Instituto Tecnológico Autónomo de México (ITAM), Otoño 2026
Profesor: Dr. J. Octavio Gutiérrez García · 20 de octubre de 2026
Integrantes: Mario Guillen Blanco · Gabriel Navarro Cerón

Resumen

Este documento presenta la evaluación experimental de desempeño de las tres versiones (Serial, P1 y P2) descritas en el documento *Descripción del código y estrategias de paralelización*. Evaluamos 2,320 ejecuciones (ocho tamaños de 20,000 a 200,000 puntos; 1, 6, 12 y 24 hilos; k de 2 a 64; diez repeticiones) en un Apple M4 Pro de 12 núcleos. Todas reprodujeron exactamente las clases y los conteos de vecinos de la versión serial. Con 200,000 puntos y 12 hilos, P1 fue 8.29 veces más rápido que la versión serial y P2 con $k = 64$, 47.77 veces; con $k \leq 4$, en cambio, P2 fue más lento que P1.

1. Metodología experimental

Datos. Los ocho conjuntos se generaron con la primera celda de la libreta del profesor,

```
make_blobs(n_samples=N, centers=4, cluster_std=0.06, random_state=11, center_box=(0, 1.0)),
```

seguida de intercambio de columnas, valor absoluto y redondeo a tres decimales. Cada N se generó por separado. Los puntos ocupan cerca de $[0, 1.0] \times [0, 0.7]$ y son muy densos: con $N = 200{,}000$ hay 199,902 en C_1, 77 en C_2 y 21 de ruido.

Matriz experimental. $N \in \{20, 40, 80, 120, 140, 160, 180, 200\} \times 10^3$; hilos $T \in \{1, 6, 12, 24\}$ (uno, la mitad, todos y el doble de los 12 núcleos); $k \in \{2, 4, 8, 16, 32, 64\}$, solo para P2; diez repeticiones. Serial $8 \times 10 = 80$, P1 $8 \times 4 \times 10 = 320$ y P2 $8 \times 4 \times 6 \times 10 = 1{,}920$: **2,320 ejecuciones** en 232 configuraciones.

Medición del tiempo. El tiempo oficial lo mide el programa con `omp_get_wtime()`:

$$T_{\text{total}} = T_{\text{build}} + T_{\text{paso1}} + T_{\text{paso2}}, \tag{1}$$

con $T_{\text{build}} = 0$ en Serial y P1 y, en P2, toda la construcción de regiones y halos. Se excluyen la lectura y escritura de CSV (leer 200,000 puntos toma unos 14 ms), la creación del equipo de hilos y la reserva de memoria.

Protocolo. Un script de Python lanzó un proceso por ejecución, una a la vez, tras una referencia serial por tamaño (excluida) cuyas huellas coincidieron con las validadas. En cada repetición r recorrió las 232 configuraciones en orden pseudoaleatorio (semilla $20261020 \cdot 1000 + r$), para que una deriva térmica o de carga afectara a todas por igual, y comparó cada ejecución con la referencia de su tamaño; ninguna falló ni se repitió. Sin variables `OMP_*` en el entorno, todas usaron el mismo ejecutable y los mismos datos. La sesión duró unos 33 minutos (19–20 de octubre de 2026, de 23:31 a 00:03, hora del centro de México), con la computadora conectada a la corriente.

Hardware y software. Los 12 núcleos de la Tabla 1 son heterogéneos y sin hilos simultáneos (SMT): 24 hilos implican *sobresuscripción*, dos por núcleo, y macOS no permite fijar hilos a núcleos. Al iniciar, el registro muestra al proceso del sistema `BTLEServer` usando el 99.6 % de un núcleo (como en las pruebas previas) y abiertos el editor y Claude Code (19 % y 9 % de un núcleo); lo tratamos como limitación y posible fuente de variación, no como causa demostrada.

Estadística. Por configuración se calculan la media $\overline{T}$, la desviación estándar muestral y el coeficiente de variación (CV) de las diez repeticiones; las aceleraciones son razones de medias, no medias de cocientes:

$$S_{P1}(N, T) = \frac{\overline{T}_{\text{serial}}(N)}{\overline{T}_{P1}(N, T)}, \qquad S_{P2}(N, k, T) = \frac{\overline{T}_{\text{serial}}(N)}{\overline{T}_{P2}(N, k, T)}. \tag{2}$$

Tabla 1: Hardware y software.

Equipo	MacBook Pro (Mac16,8); Apple M4 Pro, 12 núcleos (8 de rendimiento y 4 de eficiencia), sin SMT; 24 GB; macOS 26.5.2 (25F84), Darwin 25.5.0, arm64
Compilador y OpenMP	Apple clang 21.0.0 (clang-2100.1.1.101) con libomp 23.1.3 de Homebrew; `_OPENMP` = 202011 (OpenMP 5.1)
Banderas	`-std=c++17 -O3 -Wall -Wextra -ffp-contract=off -Xpreprocessor -fopenmp`
Código y análisis	`dbscan.cpp` (versión `m5`); Python 3.11.9, NumPy 2.4.6, Matplotlib 3.11.2

El CV mediano fue 0.55 %; 15 de las 232 configuraciones superaron el 5 %, 14 de ellas con tiempos menores de 0.15 s (la mayor: P1, $N = 140{,}000$, 12 hilos, 0.535 s, CV de 6.7 %). Tres repeticiones quedaron a más de 1.25 veces la mediana de su configuración; no se eliminó ninguna.

2. Resultados

2.1. Verificación funcional y visualización de los resultados

Corrección. Las 2,320 ejecuciones dieron huellas de clases y de conteos idénticas a la referencia serial. La validación previa incluyó casos de clases conocidas, conteos iguales a una referencia en NumPy, cero diferencias con scikit-learn y la detección de versiones alteradas a propósito. Es evidencia experimental para estas entradas; el argumento general está en la sección 2.3 del documento *Descripción del código y estrategias de paralelización*.

Compatibilidad con la libreta del curso. Para comprobar el requisito de entrada y salida ejecutamos la primera celda de la libreta original `DBSCAN_noise.ipynb` con `n_points = 4000`; el archivo `4000_data.csv` que genera es idéntico, byte por byte, al conjunto de 4,000 puntos del proyecto. El mismo ejecutable de las mediciones oficiales (versión `m5`, compilado con el `Makefile` y las banderas de la Tabla 1) lo procesó con P2, $k = 64$ y 12 hilos:

```
dbscan --input 4000_data.csv --mode p2 --k 64 --threads 12 \
       --eps 0.03 --min-samples 10 --output 4000_results.csv
```

El archivo `4000_results.csv` tiene 4,000 filas y tres columnas sin encabezado, `x,y,etiqueta`, con las coordenadas en el mismo orden y con el mismo texto que la entrada. La etiqueta es 1 si el punto no es ruido (core de primer o de segundo grado) y 0 si es ruido, la convención de `np.where(clusters==-1, 0, 1)` de la libreta.

Después ejecutamos la segunda celda de la libreta original sin cambiar su lógica; solo agregamos `plt.savefig` para guardar la imagen. La celda leyó el archivo con su instrucción original,

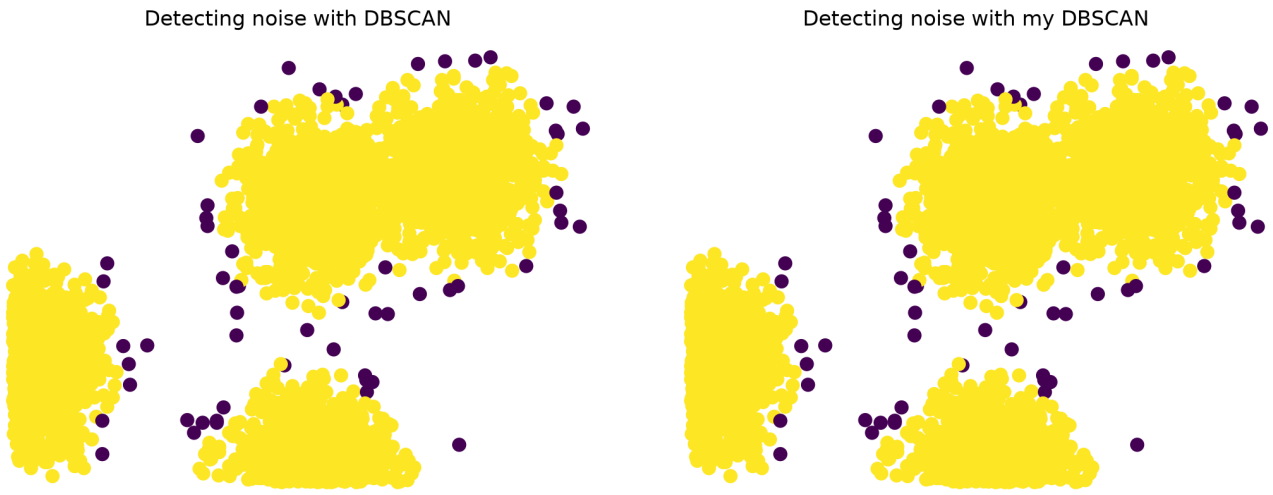
```
result = np.loadtxt(str(n_points)+"_results.csv", delimiter=",")
```

y produjo la Figura 1b. Las 4,000 etiquetas coinciden punto por punto con las de la versión Serial, cuyo archivo de salida es idéntico byte por byte, y con las que calcula scikit-learn en la primera celda (Figura 1a): en los tres casos hay 59 puntos de ruido y 3,941 que no lo son. Los archivos de entrada y salida, las imágenes y la copia ejecutada de la libreta están en la carpeta `visualizacion/`.

2.2. Desempeño

Aceleración frente al número de puntos. Ambas versiones aceleran en todos los tamaños y superan con amplitud el mínimo de 1.5 (Figura 2): P1 va de 5.99 ($N = 20{,}000$) a 8.29 ($N = 200{,}000$) y P2, de 29.92 a 47.77, entre 5.0 y 6.2 veces más rápido que P1 en cada tamaño. P1 se estabiliza desde 80,000 puntos y P2 desde 120,000; con menos, cada ejecución paralela dura milisegundos (P2: 2.9 ms con $N = 20{,}000$) y es de esperar que pesen más los costos fijos.

Tiempo frente al número de puntos. Con $N = 200{,}000$ (Figura 3) Serial tarda 8.68 s, P1 1.05 s y P2 0.18 s. Al multiplicar N por 10 el tiempo serial se multiplica por 99, consistente con los N^2 pares del Paso 1, aunque ocho tamaños no bastan para afirmar una complejidad asintótica.



(a) Primera celda: DBSCAN de scikit-learn (b) Segunda celda: `4000_results.csv` (C++)

Figura 1: Visualización con la libreta original del curso, $N = 4{,}000$ ($\varepsilon = 0.03$, `min_samples` $= 10$). Morado: ruido (etiqueta 0); amarillo: no ruido (etiqueta 1). Los títulos en inglés son los de la libreta. Las dos gráficas coinciden: los mismos 59 puntos son ruido.

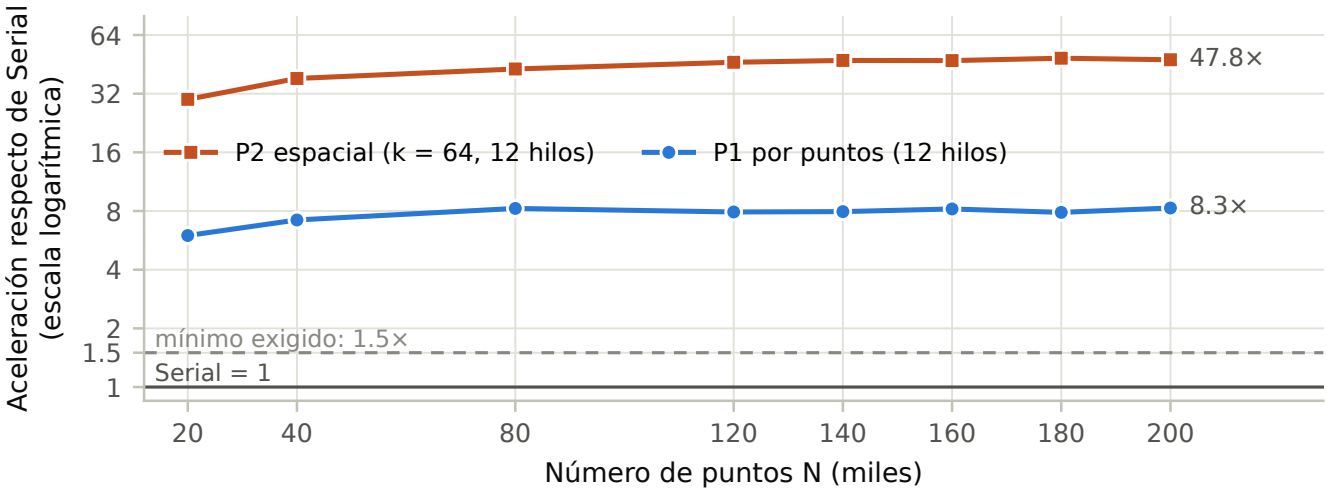


Figura 2: Aceleración respecto de Serial al crecer N, con configuraciones fijas: P1 con 12 hilos y P2 con $k = 64$ y 12 hilos. Cada punto es una razón de medias de diez repeticiones (2). El eje vertical es logarítmico para que la línea base (Serial = 1) y el mínimo exigido (1.5) se lean junto a valores cercanos a 50.

Escalamiento con hilos. P1 con un hilo tarda lo mismo que Serial (razón 0.999), así que su aceleración es puramente paralela (Figura 4a, Tabla 2): $\Pi = 5.78$ con 6 hilos (eficiencia 0.96), 8.28 con 12 y 8.47 con 24. De 6 a 12 hilos se gana mucho menos que de 1 a 6. Hipótesis compatibles, no demostradas: (a) los cuatro núcleos de eficiencia son más lentos y, con `schedule(static)`, cada hilo recibe el mismo número de puntos, así que los lentos marcan el final (compatible con que `dynamic,64` fuera un 5 % más rápido en la prueba previa); (b) `BTLEServer` ocupaba casi un núcleo; (c) recursos compartidos. Con 24 hilos no hay más núcleos; la ligera mejora podría venir de bloques más chicos que el sistema operativo reequilibra.

P2 con $k = 64$ escala peor ($\Pi = 6.10$ con 12 hilos), y la Figura 4b muestra por qué: con $k \leq 16$, Π es igual con 6, 12 y 24 hilos y queda a 2 % o menos de la cota que impone la región más costosa (1.90 frente a 1.92 con $k = 2$; 2.92 frente a 2.97 con $k = 8$): una región es una tarea indivisible. Con $k = 64$, 12 hilos alcanzan el 90 % de la cota (6.79); con 6 hilos, $\Pi = 4.12$ frente a 5.95, señal de que los supuestos del modelo (tiempo proporcional a los pares, núcleos idénticos) no se cumplen del todo.

Efecto del número de regiones. Con $N = 200{,}000$ y 12 hilos (Figura 5), P2 pasa de 3.18 ($k = 2$) y 5.29 ($k = 4$), por debajo de P1 (8.29), a 11.80, 13.90, 41.84 y 47.77 con $k = 8$, 16, 32 y 64. Más regiones

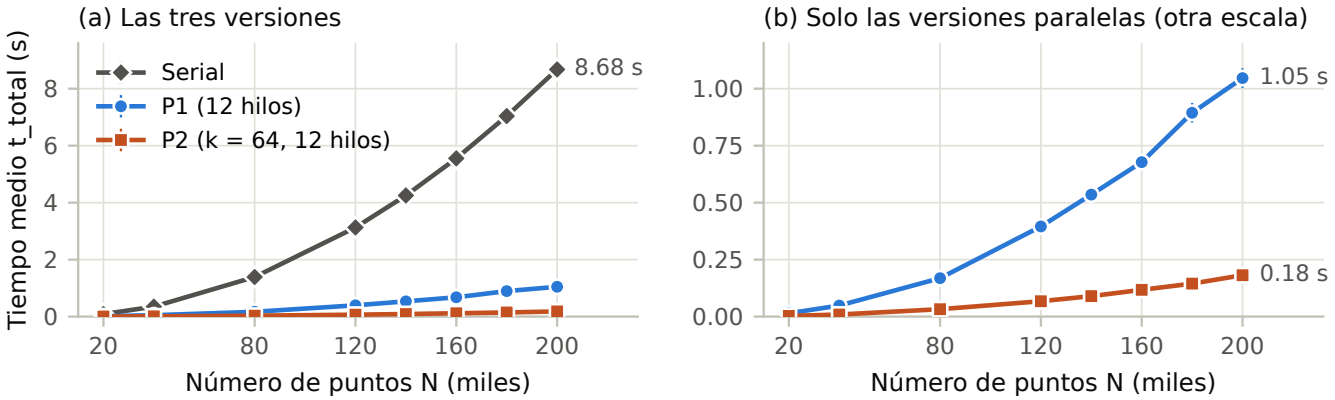


Figura 3: Tiempo medio T_{total} frente a N con las configuraciones de la Figura 2. El panel (b) repite P1 y P2 en otra escala para distinguirlos. Las barras de error (± 1 desviación estándar) casi no se ven porque son pequeñas.

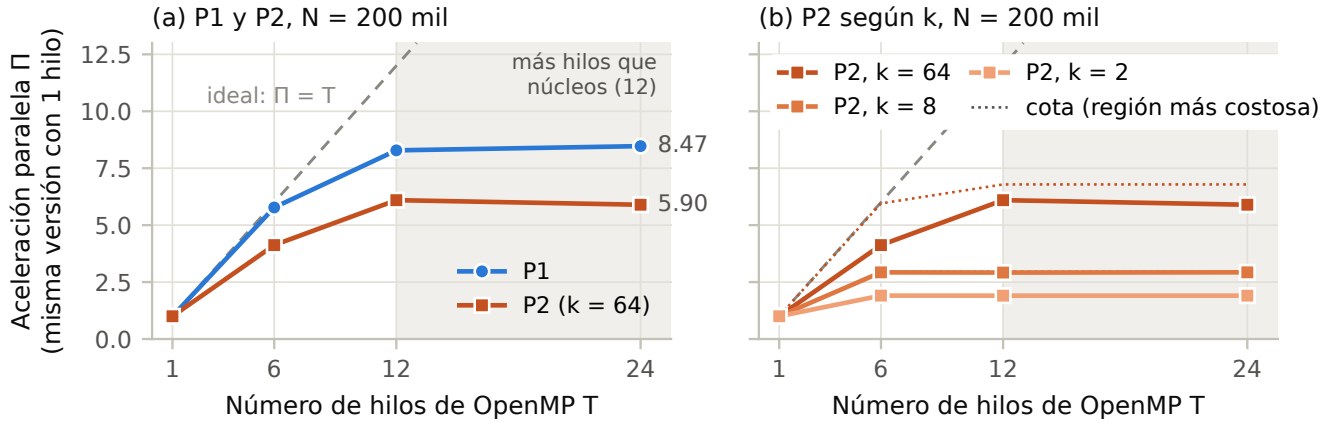


Figura 4: Escalamiento con hilos para $N = 200{,}000$, medido con la aceleración paralela $\Pi(T) = \overline{T}(1)/\overline{T}(T)$ de cada versión contra sí misma con un hilo (P2 con k fijo), que aísla el efecto de los hilos. Línea discontinua: referencia ideal $\Pi = T$; zona gris: más hilos que núcleos. En (b), las líneas punteadas son la cota que impone la región más costosa si el tiempo fuera proporcional a los pares y los núcleos fueran idénticos (Sección 3).

actúan por dos vías: menos pares (de $N^2/1.67$ a $N^2/8.13$) y más tareas, cada una más ligera (la más costosa pasa del 52 % al 14.7 % de los pares). La segunda explica el salto de $k = 16$ a $k = 32$: en la cuadrícula 4×4 una celda conserva el 36.5 % del trabajo —más que con $k = 8$, por eso Π baja de 2.92 a 2.72— y en la de 4×8 la más pesada baja al 14.5 %. Aumentar k no garantiza una mejora: depende de dónde caen las fronteras respecto de los grupos, y el halo (de 38,791 copias de puntos con $k = 2$ a 247,114 con $k = 64$) y la construcción, $O(kN)$, crecen con k (no medimos $k > 64$). La Figura 6 repite el patrón con 20,000, 120,000 y 200,000 puntos: $k = 2$ siempre queda debajo de P1, $k = 8$ por encima, y la ventaja de $k = 64$ crece con N (29.9, 46.4 y 47.8).

Por qué P2 superó a P1, y cuándo no. P1 hace las mismas N^2 comparaciones que Serial y las reparte entre hilos; P2 primero elimina comparaciones con restricciones espaciales y luego reparte las restantes. Con $N = 200{,}000$ y 12 hilos, P2 escala peor ($\Pi = 6.10$ contra 8.28), pero evalúa 8.13 veces menos pares y termina 5.76 veces antes. P2 fue más lento que P1 en 48 de las 192 combinaciones (N, k, T) comparables: exactamente las que tienen $k \in \{2, 4\}$ y $T \geq 6$. Ahí hay a lo más cuatro tareas, la más pesada con cerca de la mitad del trabajo ($\Pi \leq 2.04$), y el ahorro de pares (1.67 o 2.63) no compensa la aceleración de P1 con 6 o más hilos (5.6 a 8.5). Con un hilo, P2 superó a Serial en todas las configuraciones (factor mínimo 1.65). Los mejores resultados fueron 8.53 para P1 ($N = 180{,}000$, 24 hilos) y 48.64 para P2 ($N = 180{,}000$, $k = 64$, 12 hilos).

Tabla 2: Escalamiento con hilos para $N = 200{,}000$ (Serial: 8.6758 ± 0.0161 s, CV 0.2 %). Π: aceleración paralela contra la misma versión con un hilo; $E = \Pi/T$: eficiencia; S: aceleración respecto de Serial.

	P1				P2, $k = 64$			
T	media ± desv. (s)	CV	Π	E	media ± desv. (s)	CV	Π	S
1	8.6696 ± 0.0362	0.4 %	1.00	1.00	1.1074 ± 0.0032	0.3 %	1.00	7.83
6	1.5011 ± 0.0143	1.0 %	5.78	0.96	0.2685 ± 0.0005	0.2 %	4.12	32.31
12	1.0466 ± 0.0433	4.1 %	8.28	0.69	0.1816 ± 0.0042	2.3 %	6.10	**47.77**
24	1.0236 ± 0.0247	2.4 %	8.47	0.35	0.1878 ± 0.0036	1.9 %	5.90	46.19

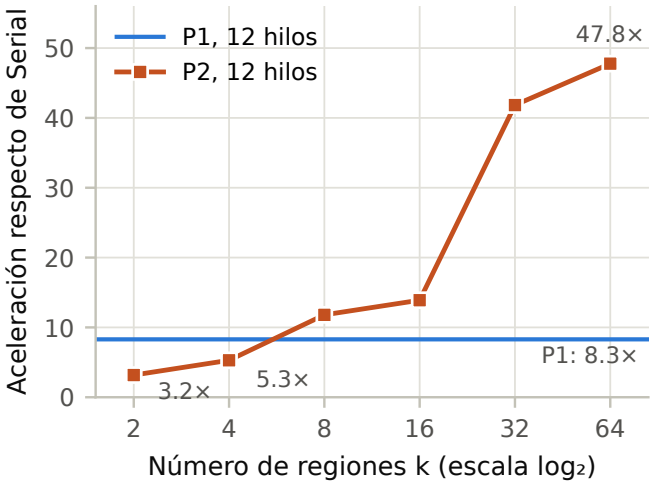


Figura 5: Efecto de k con $N = 200{,}000$ y 12 hilos: aceleración de P2 respecto de Serial; la línea azul es P1 con los mismos 12 hilos.

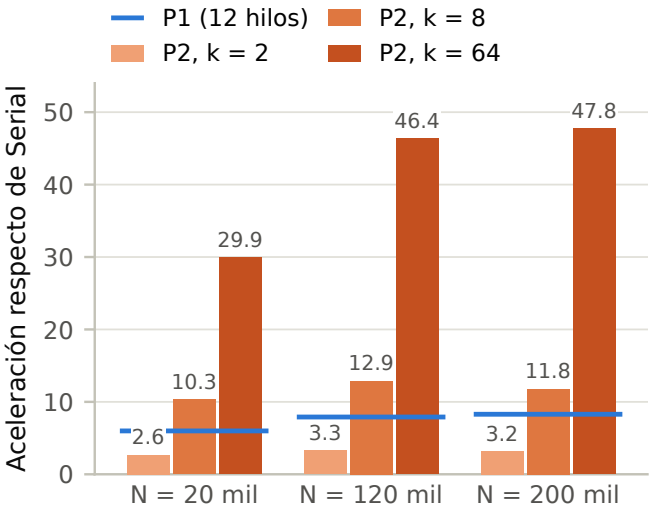


Figura 6: Interacción entre N y k con 12 hilos: aceleración de P2 con un k pequeño, uno intermedio y uno grande; los segmentos azules marcan a P1 con 12 hilos.

3. Ley de Amdahl y descomposición del speedup

La ley de Amdahl [4, 2] acota la aceleración de un trabajo *fijo* cuya fracción f se divide perfectamente entre p procesadores idénticos, mientras el resto es secuencial:

$$S_{\text{Amdahl}}(p) = \frac{1}{(1-f) + f/p} < \frac{1}{1-f}. \tag{3}$$

Supone además que repartir no cuesta nada; la sincronización, el desbalance de carga y la contención reducen todavía más la aceleración real [2].

La aceleración total de P2, 47.77 con 12 hilos, no contradice (3): compara dos algoritmos que hacen distinta cantidad de trabajo. La comparación legítima separa los dos efectos:

$$A(N,k) = \frac{\overline{T}_{\text{serial}}(N)}{\overline{T}_{P2}(N,k,1)}, \qquad \Pi(N,k,T) = \frac{\overline{T}_{P2}(N,k,1)}{\overline{T}_{P2}(N,k,T)}, \qquad S_{\text{total}} = A \cdot \Pi. \tag{4}$$

A es la ganancia algorítmica (P2 con un hilo frente a Serial) y Π la ganancia paralela de la misma estrategia; solo Π debe compararse con Amdahl. Con $N = 200{,}000$, $k = 64$ y $T = 12$: $A = 7.83$, $\Pi = 6.10$ y $S_{\text{total}} = 47.77$. A es cercano a la reducción de pares, $N^2/\sum_r n_r|L_r| = 8.13$; la diferencia se debe casi toda a la construcción (3.5 % del tiempo con un hilo), porque sin ella A sería 8.12. Y $\Pi = 6.10 < 12$, como exige Amdahl. La Tabla 3 muestra la descomposición para cada k.

La región más costosa como parte no divisible. En P2 cada región es una tarea indivisible. Si el tiempo de una región fuera proporcional a sus pares y los núcleos fueran idénticos, ningún reparto terminaría antes que la región más pesada, sin importar el número de hilos:

$$\Pi \le \frac{\sum_r n_r|L_r|}{\max_r n_r|L_r|} = \frac{1}{s_{\max}}. \tag{5}$$

Tabla 3: Descomposición $S_{\text{total}} = A \cdot \Pi$ con $N = 200{,}000$ y 12 hilos. N^2/pares: reducción de pares del Paso 1; $1/s_{\text{máx}}$: cota de Π por la región más costosa, que tiene la fracción $s_{\text{máx}}$ (`max_task_share`) de los pares.

k	regiones con puntos	N^2/pares	A	Π (12 hilos)	$1/s_{\text{máx}}$	S_{total}
2	2	1.67	1.67	1.90	1.92	3.18
4	4	2.63	2.62	2.02	2.04	5.29
8	8	4.06	4.04	2.92	2.97	11.80
16	15	5.15	5.10	2.72	2.74	13.90
32	29	6.47	6.35	6.59	6.88	41.84
64	55	8.13	7.83	6.10	6.79	47.77

Tabla 4: Prueba complementaria de sensibilidad: 40,000 puntos, 12 hilos, media de tres repeticiones (mediciones provisionales). Con datos uniformes P2 tarda unos 3 ms, así que esa cifra es poco precisa.

Distribución	Serial (s)	P1	P2, $k = 64$	N^2/pares	$s_{\text{máx}}$
Uniforme	0.3366	8.0	121.9	36.38	0.020
Desbalanceada	0.3417	8.4	7.0	4.08	0.566
Concentrada	0.3398	7.9	1.0	1.01	1.000

Es una versión del cuello de botella de Amdahl: la región más pesada hace el papel de la parte que no se puede dividir. Es un modelo idealizado, no una identidad (supone costo uniforme por par, núcleos iguales y reparto gratuito). Para la Figura 4b lo evaluamos reconstruyendo, con la regla del programa, el costo de cada región a partir de los datos (la suma coincide exactamente con `pairs_step1`) y simulando el reparto de mayor a menor.

P1 y la fracción paralelizable. Si se despeja f de (3) con los Π de P1, resulta $f = 0.992$ con 6 hilos y $f = 0.959$ con 12. Con una parte secuencial fija, f sería la misma; no lo es, así que la pérdida depende del hardware —núcleos heterogéneos, que no equivalen a doce procesadores idénticos— más que de una parte secuencial del código. Con estas mediciones no podemos obtener una fracción serial exacta ni atribuir toda la pérdida de eficiencia a un solo factor.

4. Limitaciones y prueba de sensibilidad

Los resultados corresponden a una máquina y a una familia de datos: cuatro grupos gaussianos muy densos (99.9 % de los puntos en C_1), lo cual favorece al particionamiento espacial. Para dimensionarlo hicimos una prueba complementaria, *fuera* de las 2,320 ejecuciones oficiales, con datos sintéticos: uniformes en $[0, 1.3]^2$; "desbalanceados", con el 80 % de los puntos en uno de cinco grupos; y "concentrados", con el 99.5 % en un cuadrado de 0.06×0.06 y pocos puntos lejanos que estiran la caja. Se midieron tres repeticiones con 12 hilos con una compilación previa a la integración final, cuyo modo espacial es el algoritmo de P2; las clases y conteos coincidieron con Serial, también con el ejecutable oficial (Tabla 4).

Las regiones de P2 tienen el mismo tamaño, no el mismo número de puntos. Si casi todos caen en una, apenas se ahorran comparaciones, una tarea concentra el trabajo y la aceleración desaparece: con datos concentrados P2 quedó en 1.0 mientras P1 conservó unas ocho veces, y en un caso adverso P2 podría ser más lento que Serial porque además paga la construcción. El P2 por bloques de Mario mantuvo unas ocho veces en las tres distribuciones, porque no depende de la geometría. Otras limitaciones: ε y m fijos (un ε mayor ensancha los halos), hilos sin fijar a núcleos, `BTLEServer` activo y un tiempo que excluye la lectura.

5. Uso responsable y ético de IA generativa

Seguimos las pautas de IEEE [7] (declarar la herramienta, las partes del trabajo en que se usó y el nivel de uso) y de Elsevier [8] (la herramienta no es autora; los autores responden por el contenido).

Herramienta y tareas. Usamos Claude Opus 5.5 (Anthropic), mediante Claude Code, del 18 al 20 de octubre de 2026. Del 18 al 19 de octubre la usamos para: interpretación del enunciado y plan de arquitectura; implementación asistida de `dbscan.cpp` (Serial, CSV, P1 y P2 espacial), del `Makefile`, de

la validación, del ejecutor y del análisis; adaptación y validación del P2 de Mario; y documentación. El 20 de octubre se usó además para redactar este reporte y generar sus figuras con un script desde el CSV oficial. No se usó ninguna otra herramienta de IA generativa.

Aportación del equipo. El equipo aprobó el plan tras dos rondas de revisión, generó los datos con la libreta, revisó cada etapa antes de integrarla, decidió qué P2 entregar y ejecutó el experimento oficial en su computadora. Mario diseñó e implementó la primera P2; su autoría se reconoce en el código, en este reporte y en las comparaciones del equipo.

Verificación. Ninguna salida de la IA se aceptó por su apariencia: las versiones se contrastaron con la serial (huellas idénticas en las 2,320 ejecuciones), con casos de clases conocidas, con una referencia en NumPy y con scikit-learn; se introdujeron errores a propósito para comprobar que las pruebas los detectan; una revisión del código, también asistida por IA, verificó escrituras disjuntas y barreras; y las cifras del reporte se recalcularon desde los datos de las mediciones.

Declaración. Durante la preparación de este trabajo, el equipo usó Claude Opus 5.5 para las tareas descritas. Después de usarla, el equipo revisó y editó el contenido según fue necesario y asume la responsabilidad total por la corrección del código, las decisiones de diseño, la interpretación de los experimentos y el contenido de este reporte. Ambos integrantes revisaron el reporte completo.

6. Conclusiones

Las dos versiones paralelas reprodujeron exactamente la clasificación serial —clases y conteos de vecinos— en las 2,320 ejecuciones y superaron con amplitud el mínimo de 1.5. P1, la paralelización directa del algoritmo exhaustivo, escala casi idealmente hasta 6 hilos y llega a 8.29 con 12; su pérdida de eficiencia posterior es compatible con la heterogeneidad de los núcleos, aunque no pudimos aislar su causa. P2 fue el más rápido con los datos oficiales (47.77 con $N = 200{,}000$, $k = 64$ y 12 hilos) porque combina dos ganancias: evalúa 8.13 veces menos pares gracias a halos que no pierden vecinos, y reparte las regiones entre hilos ($\Pi = 6.10$). Esa ventaja tiene condiciones: con $k \leq 4$ P2 fue más lento que P1, su escalamiento está acotado por la región más costosa y, con datos concentrados, su aceleración desapareció. P2 conviene cuando los puntos se reparten entre muchas regiones, como en los datos del curso; P1 ofrece un rendimiento más predecible, independiente de la distribución.

Datos experimentales y reproducibilidad. Todos los datos del experimento están en el archivo `datos_experimentos_DBSCAN.xlsx`, que acompaña a este documento: la hoja *Mediciones* tiene las 2,320 ejecuciones una por una; la hoja *Resumen*, el promedio de las diez iteraciones, la desviación estándar, el speedup y la eficiencia de cada configuración; y las hojas *Speedup P1* y *Speedup P2* las tablas de aceleración. El código fuente de las tres versiones está en `dbscan.cpp`.

Referencias

[1] J. O. Gutiérrez García. *Proyecto Apertura 2026B: DBSCAN* (enunciado) y libreta `DBSCAN_noise.ipynb`. ITAM, 2026.

[2] J. O. Gutiérrez García. *Introducción y arquitecturas paralelas* (diapositivas del curso), ITAM, 2026.

[3] M. Ester, H.-P. Kriegel, J. Sander y X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. En *Proc. KDD-96*, pp. 226–231, 1996.

[4] G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. En *AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.

[5] R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM J. Appl. Math.*, 17(2):416–429, 1969.

[6] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *JMLR*, 12:2825–2830, 2011.

[7] IEEE Robotics and Automation Society. *Guidelines for Generative AI Usage.* `https://www.ieee-ras.org/publications/guidelines-for-generative-ai-usage`.

[8] Elsevier. *Generative AI policies for journals.* `https://www.elsevier.com/about/policies-and-standards/generative-ai-policies-for-journals`.